

SUBMITTED BY: HARSHPREET KAUR (harshpreet.kaur@kenexai.com)

DATE OF SUBMISSION: 5 JUNE, 2026

› Question 1: Income Tax Calculator (Basic Operations)

↳ 2 cells hidden

› Question 2: Student Final Grade Calculator (Global & Local Variables)

↳ 2 cells hidden

› Question 3: String Slicing Operations

↳ 2 cells hidden

› Question 4: Collection Types - Inventory Management System

↳ 2 cells hidden

✓ Question 5 Grade Classification System (Conditions)

Description

Create a comprehensive grade classification system that uses conditional statements to categorize student performance.

Requirements

1. Input student marks for 5 subjects
2. Calculate average marks
3. Classify performance using if-elif-else:

- Excellent: 90% and above
 - Very Good: 80-89%
 - Good: 70-79%
 - Average: 60-69%
 - Below Average: 50-59%
 - Fail: Below 50%
4. Check if student passed ($\geq 50\%$) using logical operators
 5. Display detailed report with all subject marks, average, percentage, and classification

Input/Output Example

Enter marks for Math: 92

Enter marks for Science: 88

Enter marks for English: 85

Enter marks for History: 90

Enter marks for Geography: 87

=== Student Performance Report ===

Math: 92

Science: 88

English: 85

History: 90

Geography: 87

Average Marks: 88.4

Percentage: 88.4%

Classification: Very Good

Status: Passed

Constraints

- Each mark should be between 0 and 100
- Use proper conditional logic (if-elif-else)
- Validate all inputs

```
def get_valid_marks(subject):  
    while True:
```

```
try:
    marks = float(input(f"Enter marks for {subject}: "))

    if 0 <= marks <= 100:
        return marks
    else:
        print("Marks must be between 0 and 100.")

except ValueError:
    print("Invalid input! Please enter a numeric value.")

math = get_valid_marks("Math")
science = get_valid_marks("Science")
english = get_valid_marks("English")
history = get_valid_marks("History")
geography = get_valid_marks("Geography")

# Calculate average and percentage
total_marks = math + science + english + history + geography
average = total_marks / 5
percentage = average

# Classification using if-elif-else
if percentage >= 90:
    classification = "Excellent"
elif percentage >= 80:
    classification = "Very Good"
elif percentage >= 70:
    classification = "Good"
elif percentage >= 60:
    classification = "Average"
elif percentage >= 50:
    classification = "Below Average"
else:
    classification = "Fail"

# Pass/Fail using logical operators
if percentage >= 50 and math >= 0 and science >= 0 and english >= 0 and history >= 0:
    status = "Passed"
else:
    status = "Failed"

print("\n=== Student Performance Report ===")

print(f"Math: {math}")
print(f"Science: {science}")
print(f"English: {english}")
print(f"History: {history}")
print(f"Geography: {geography}")

print(f"\nAverage Marks: {average:.2f}")
```

```
print(f"Percentage: {percentage:.2f}%")
print(f"Classification: {classification}")
print(f"Status: {status}")
```

```
Enter marks for Math: 92
Enter marks for Science: 85
Enter marks for English: 88
Enter marks for History: 90
Enter marks for Geography: 87
```

```
=== Student Performance Report ===
```

```
Math: 92.0
Science: 85.0
English: 88.0
History: 90.0
Geography: 87.0
```

```
Average Marks: 88.40
Percentage: 88.40%
Classification: Very Good
Status: Passed
```

✓ Question 6 Password Validation System (Iterations)

Description

Create a password validation system with multiple attempts using while loops.

Requirements

1. Set a correct password (e.g., "SecurePass123!")
2. Allow user maximum 3 attempts to enter the correct password
3. Use a while loop to handle attempts
4. After each incorrect attempt, show remaining attempts
5. If password is correct, grant access
6. If all attempts are exhausted, lock the account
7. Use break statement when password is correct
8. Use continue to skip invalid attempts (empty input)

Input/Output Example

```
=== Password Validation System ===
```

```
Enter password: wrongpass
```

```
Incorrect password. 2 attempts remaining.
```

```
Enter password: Password cannot be empty. 2 attempts remaining.
```

Enter password: SecurePass123!

Access granted! Welcome.

--- OR ---

Enter password: wrong1

Incorrect password. 2 attempts remaining.

Enter password: wrong2

Incorrect password. 1 attempt remaining.

Enter password: wrong3

Maximum attempts exceeded. Account locked.

Constraints

- Maximum 3 attempts
- Handle empty inputs
- Use while loop with break and continue

```
CORRECT_PASSWORD = "SecurePass123!"
MAX_ATTEMPTS = 3
attempts = 0
print("=== Password Validation System ===")

while attempts < MAX_ATTEMPTS:
    password = input("Enter password: ")
    if password == "":
        remaining = MAX_ATTEMPTS - attempts
        print(f"Password cannot be empty. {remaining} attempts remaining.")
        continue
    if password == CORRECT_PASSWORD:
        print("Access granted! Welcome.")
        break
    attempts += 1 #incorrect
    remaining = MAX_ATTEMPTS - attempts

    if remaining > 0:
        print(f"Incorrect password. {remaining} attempt(s) remaining.")
    else:
        print("Maximum attempts exceeded. Account locked.")
```

```
=== Password Validation System ===
Enter password: 1234
Incorrect password. 2 attempt(s) remaining.
Enter password:
Password cannot be empty. 2 attempts remaining.
Enter password: 567
```

```
Incorrect password. 1 attempt(s) remaining.  
Enter password: SecurePass123!  
Access granted! Welcome.
```

> Question 7: Data Processing with Loops (ETL Simulation)

↳ 2 cells hidden

> Question 8: Pattern Matching - Data Format Processor

↳ 2 cells hidden

> Question 9: String Manipulation - Text Analyzer

↳ 2 cells hidden

> Question 10: Comprehensive Calculator with All Concepts

↳ 2 cells hidden